Red Neuronal - UnexpoDancing

Carlos Urbina EXP.2022100156, Vanessa Bolivar EXP.2022100162. GRUPO 2. Inteligencia Artificial

01 de Marzo de 2026

Table of Contents

Red Neuronal Artificial (TensorFlow - Keras) 1
1. Introducción 1
3. Carga del Dataset 2
4. Preparación de los Datos 2
5. División en Entrenamiento y Prueba 2
6. Separación de Variables 2
7. Normalización 2
8. Codificación One-Hot (Manual) 2
9. Construcción del Modelo 2
10. Compilación del Modelo 2
11. Entrenamiento 3
12. Evaluación del Modelo 3
13. Predicciones y Matriz de Confusión 4
14. Conclusión 5

Red Neuronal Artificial (TensorFlow - Keras)

1. Introducción

En esta sección se implementa una Red Neuronal Artificial utilizando Keras con backend TensorFlow, con el objetivo de clasificar movimientos correctos e incorrectos en el sistema tipo Just Dance, basándose en los ángulos de brazos y piernas.

2. Carga de Librerías

```
library(keras)
library(caret)
library(readr)
library(tensorflow)
```

3. Carga del Dataset

```
data <- read_csv("dance_dataset.csv")
```

4. Preparación de los Datos

```
data$label <- as.factor(data$label)
```

5. División en Entrenamiento y Prueba

```
set.seed(42)
trainIndex <- createDataPartition(data$label, p = 0.8, list = FALSE)
trainData <- data[trainIndex, ]
testData  <- data[-trainIndex, ]
```

6. Separación de Variables

```
x_train <- as.matrix(trainData[, c("left_arm", "right_arm", "left_leg",
"right_leg")])
x_test  <- as.matrix(testData[, c("left_arm", "right_arm", "left_leg",
"right_leg")])

y_train <- as.numeric(as.factor(trainData$label)) - 1
y_test  <- as.numeric(as.factor(testData$label)) - 1
```

7. Normalización

```
x_train <- scale(x_train)
x_test  <- scale(x_test,
                 center = attr(x_train, "scaled:center"),
                 scale = attr(x_train, "scaled:scale"))
```

8. Codificación One-Hot (Manual)

```
num_classes <- length(unique(y_train))
y_train_cat <- as.matrix(model.matrix(~ 0 + as.factor(y_train)))
y_test_cat  <- as.matrix(model.matrix(~ 0 + as.factor(y_test)))
colnames(y_train_cat) <- NULL
colnames(y_test_cat) <- NULL
```

9. Construcción del Modelo

```
model <- keras_model_sequential()
model$add(layer_input(shape = c(4)))
model$add(layer_dense(units = 16, activation = "relu"))
model$add(layer_dense(units = 8, activation = "relu"))
model$add(layer_dense(units = num_classes, activation = "softmax"))
```

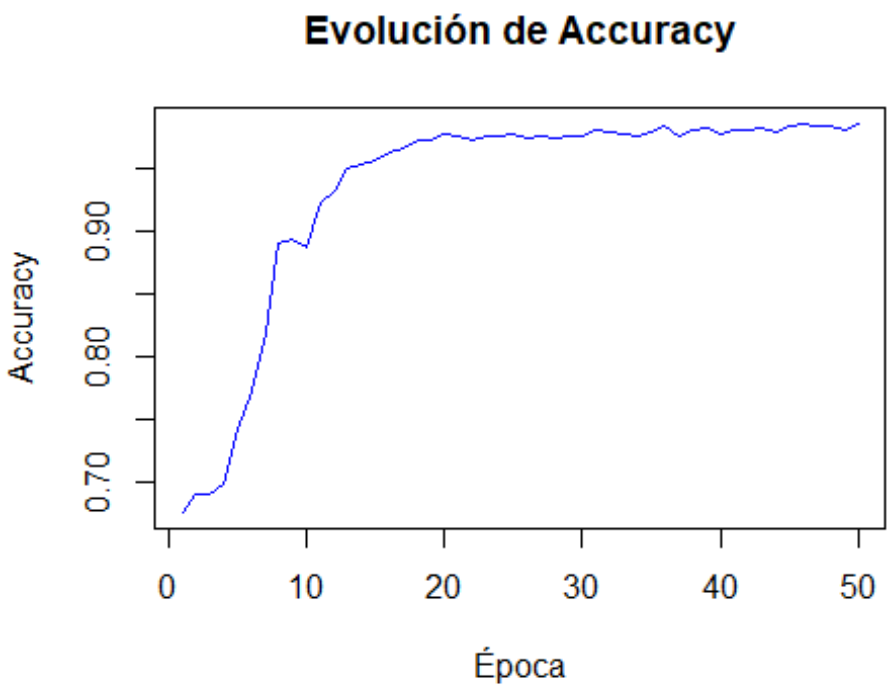
10. Compilación del Modelo

```
model$compile(
  loss = "categorical_crossentropy",
  optimizer = "adam",
```

```
  metrics = list("accuracy")
)
```

11. Entrenamiento

```
history <- model$fit(
  x_train,
  y_train_cat,
  epochs = as.integer(50),
  batch_size = as.integer(16),
  validation_split = 0.2,
  verbose = 0
)

# Convertimos el historial a un formato que R pueda entender para
graficar
history_df <- as.data.frame(history$history)
plot(history_df$accuracy, type = "l", col = "blue", main = "Evolución de
Accuracy", xlab = "Época", ylab = "Accuracy")
```



12. Evaluación del Modelo

```
score <- model$evaluate(x_test, y_test_cat, verbose = 0)
print(score)

## [1] 0.05230523 0.98500001
```

13. Predicciones y Matriz de Confusión

```
predictions <- model$predict(x_test)

##
## [1m1/7[0m [32m━━[0m[37m━━━━━━━━━━━━━━━━━━━[0m [1m0s[0m 76ms/step
## [1m7/7[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m 12ms/step

predicted_classes <- apply(predictions, 1, which.max) - 1

confusionMatrix(factor(predicted_classes, levels = 0:(num_classes-1)),
                factor(y_test, levels = 0:(num_classes-1)))

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1  2
##          0 57  0  0
##          1  0 80  0
##          2  3  0 60
##
## Overall Statistics
##
##                Accuracy : 0.985
##                  95% CI : (0.9568, 0.9969)
##     No Information Rate : 0.4
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9773
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 0 Class: 1 Class: 2
## Sensitivity             0.950      1.0   1.0000
## Specificity             1.000      1.0   0.9786
## Pos Pred Value          1.000      1.0   0.9524
## Neg Pred Value          0.979      1.0   1.0000
## Prevalence              0.300      0.4   0.3000
## Detection Rate          0.285      0.4   0.3000
## Detection Prevalence    0.285      0.4   0.3150
## Balanced Accuracy       0.975      1.0   0.9893
```

14. Conclusión

La red neuronal permitió modelar relaciones no lineales entre los ángulos corporales y la clasificación del movimiento.

El modelo fue entrenado durante 50 épocas utilizando el optimizador Adam y función de pérdida categorical crossentropy.

La métrica de accuracy obtenida permite comparar el desempeño con el árbol de decisión desarrollado previamente.

Evaluación en Python

Primero debemos cargar el CSV y convertir las etiquetas de texto a números que la red neuronal pueda entender. Importamos también todas las herramientas necesarias.

In [2]:
```
# In [1]:
import pandas as pd
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
```

1. Carga y preprocesamiento del dataset

Cargamos el dataset con las posiciones de las extremidades y preparamos las etiquetas. Normalizar los datos es muy importante para el correcto funcionamiento de las redes neuronales

In [3]:
```
# In [2]:
# Cargar dataset de movimientos
data = pd.read_csv("dance_dataset.csv")

# Separar Características (X) y Etiquetas (y)
X = data[["left_arm", "right_arm", "left_leg", "right_leg"]]
y = data["label"]

# Convertir etiquetas a números
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

# Escalar datos
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Separar entrenamiento y testing
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42
)
print(f"Datos listos. Clases detectadas: {encoder.classes_}")
```

```
Datos listos. Clases detectadas: ['BUENO' 'INCORRECTO' 'PERFECTO']
```

2. Diseño de la Red Neuronal

Diseñaremos una red densa (Deep Neural Network) para aprender los patrones de baile. Usamos una capa de entrada con 4 neuronas correspondientes a los brazos y piernas, y una capa de salida con activación 'softmax' para obtener las probabilidades de las 3 clases posibles.

In [5]:
```
# In [3]:
# Definir la arquitectura de la red
model = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation='relu', input_shape=(4,)),
    tf.keras.layers.Dense(12, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

# Compilar el modelo
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

```
C:\Users\user\AppData\Local\Programs\Python\Python311\Lib\site-
packages\keras\src\layers\core\dense.py:106: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape
dense (Dense)	(None, 16)
dense_1 (Dense)	(None, 12)
dense_2 (Dense)	(None, 3)

Total params: 323 (1.26 KB)

Trainable params: 323 (1.26 KB)

Non-trainable params: 0 (0.00 B)

3. Entrenamiento del Modelo

Entrenamos y evaluamos el modelo.

In [6]:
```
# In [4]:
print("Entrenando la Red Neuronal...")
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)
```

```
Entrenando la Red Neuronal...
Epoch 1/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m1s[0m
7ms/step - accuracy: 0.3344 - loss: 1.2331 - val_accuracy: 0.4313 -
val_loss: 1.1062
Epoch 2/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.6016 - loss: 0.9795 - val_accuracy: 0.6500 -
val_loss: 0.8808
Epoch 3/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.7141 - loss: 0.7573 - val_accuracy: 0.6812 -
val_loss: 0.6686
Epoch 4/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.7906 - loss: 0.5917 - val_accuracy: 0.8375 -
val_loss: 0.5368
Epoch 5/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.7984 - loss: 0.4998 - val_accuracy: 0.8062 -
val_loss: 0.4662
Epoch 6/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.7703 - loss: 0.4482 - val_accuracy: 0.8375 -
val_loss: 0.4250
Epoch 7/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.8250 - loss: 0.4144 - val_accuracy: 0.8938 -
val_loss: 0.3975
Epoch 8/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.8656 - loss: 0.3881 - val_accuracy: 0.8875 -
val_loss: 0.3719
Epoch 9/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.8781 - loss: 0.3661 - val_accuracy: 0.8625 -
val_loss: 0.3497
Epoch 10/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.8938 - loss: 0.3455 - val_accuracy: 0.8938 -
val_loss: 0.3280
Epoch 11/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
```

```
3ms/step - accuracy: 0.9141 - loss: 0.3235 - val_accuracy: 0.9312 -
val_loss: 0.3060
Epoch 12/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.9312 - loss: 0.3014 - val_accuracy: 0.9563 -
val_loss: 0.2848
Epoch 13/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.9359 - loss: 0.2809 - val_accuracy: 0.9688 -
val_loss: 0.2657
Epoch 14/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9422 - loss: 0.2616 - val_accuracy: 0.9688 -
val_loss: 0.2451
Epoch 15/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.9563 - loss: 0.2380 - val_accuracy: 0.9625 -
val_loss: 0.2161
Epoch 16/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9594 - loss: 0.2170 - val_accuracy: 0.9688 -
val_loss: 0.1965
Epoch 17/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.9625 - loss: 0.1986 - val_accuracy: 0.9750 -
val_loss: 0.1789
Epoch 18/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9688 - loss: 0.1821 - val_accuracy: 0.9750 -
val_loss: 0.1625
Epoch 19/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9688 - loss: 0.1672 - val_accuracy: 0.9688 -
val_loss: 0.1468
Epoch 20/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9688 - loss: 0.1536 - val_accuracy: 0.9688 -
val_loss: 0.1355
Epoch 21/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9703 - loss: 0.1467 - val_accuracy: 0.9875 -
val_loss: 0.1279
Epoch 22/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9781 - loss: 0.1319 - val_accuracy: 0.9875 -
val_loss: 0.1146
Epoch 23/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9766 - loss: 0.1253 - val_accuracy: 0.9875 -
val_loss: 0.1067
Epoch 24/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
6ms/step - accuracy: 0.9750 - loss: 0.1193 - val_accuracy: 0.9875 -
```

```
val_loss: 0.1003
Epoch 25/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9812 - loss: 0.1091 - val_accuracy: 0.9875 -
val_loss: 0.0941
Epoch 26/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.9750 - loss: 0.1052 - val_accuracy: 0.9937 -
val_loss: 0.0953
Epoch 27/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9766 - loss: 0.1017 - val_accuracy: 0.9875 -
val_loss: 0.0839
Epoch 28/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
6ms/step - accuracy: 0.9781 - loss: 0.0955 - val_accuracy: 0.9875 -
val_loss: 0.0795
Epoch 29/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9828 - loss: 0.0904 - val_accuracy: 0.9937 -
val_loss: 0.0777
Epoch 30/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9844 - loss: 0.0861 - val_accuracy: 0.9875 -
val_loss: 0.0741
Epoch 31/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9797 - loss: 0.0852 - val_accuracy: 0.9875 -
val_loss: 0.0701
Epoch 32/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9781 - loss: 0.0830 - val_accuracy: 0.9875 -
val_loss: 0.0691
Epoch 33/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9812 - loss: 0.0778 - val_accuracy: 0.9937 -
val_loss: 0.0695
Epoch 34/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9797 - loss: 0.0774 - val_accuracy: 0.9875 -
val_loss: 0.0688
Epoch 35/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
6ms/step - accuracy: 0.9812 - loss: 0.0794 - val_accuracy: 0.9937 -
val_loss: 0.0651
Epoch 36/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
7ms/step - accuracy: 0.9797 - loss: 0.0738 - val_accuracy: 0.9937 -
val_loss: 0.0611
Epoch 37/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9812 - loss: 0.0729 - val_accuracy: 0.9937 -
val_loss: 0.0597
```

```
Epoch 38/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9859 - loss: 0.0692 - val_accuracy: 0.9875 -
val_loss: 0.0574
Epoch 39/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9781 - loss: 0.0682 - val_accuracy: 0.9937 -
val_loss: 0.0561
Epoch 40/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
6ms/step - accuracy: 0.9828 - loss: 0.0664 - val_accuracy: 0.9937 -
val_loss: 0.0552
Epoch 41/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9828 - loss: 0.0644 - val_accuracy: 0.9937 -
val_loss: 0.0539
Epoch 42/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9859 - loss: 0.0617 - val_accuracy: 0.9875 -
val_loss: 0.0579
Epoch 43/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9844 - loss: 0.0625 - val_accuracy: 0.9937 -
val_loss: 0.0528
Epoch 44/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9812 - loss: 0.0609 - val_accuracy: 0.9937 -
val_loss: 0.0527
Epoch 45/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9859 - loss: 0.0615 - val_accuracy: 0.9937 -
val_loss: 0.0516
Epoch 46/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9828 - loss: 0.0594 - val_accuracy: 0.9875 -
val_loss: 0.0548
Epoch 47/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9812 - loss: 0.0628 - val_accuracy: 0.9937 -
val_loss: 0.0518
Epoch 48/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
3ms/step - accuracy: 0.9812 - loss: 0.0578 - val_accuracy: 0.9875 -
val_loss: 0.0555
Epoch 49/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9875 - loss: 0.0575 - val_accuracy: 0.9937 -
val_loss: 0.0484
Epoch 50/50
[1m40/40[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
4ms/step - accuracy: 0.9844 - loss: 0.0574 - val_accuracy: 0.9937 -
val_loss: 0.0482
```

4. Evaluación y Predicciones

Generamos una matriz de confusión para observar la precisión real del modelo clasificando los movimientos.

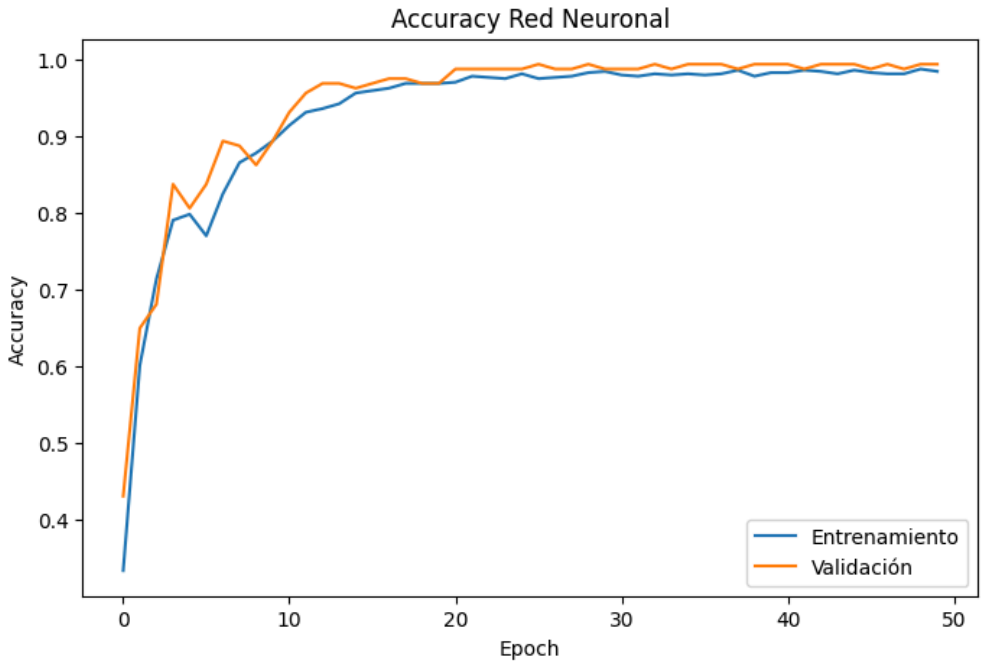
In [7]:
```
# In [5]:
# Evaluar precisión final
loss, accuracy = model.evaluate(X_test, y_test)
print(f"\nAccuracy Red Neuronal: {accuracy * 100:.2f}%")

# Predicciones
y_pred = np.argmax(model.predict(X_test), axis=1)

print("\nMatriz de Confusión:")
print(confusion_matrix(y_test, y_pred))

print("\nReporte de Clasificación:")
print(classification_report(y_test, y_pred, target_names=encoder.classes_))
```

```
[1m7/7[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
5ms/step - accuracy: 0.9800 - loss: 0.0547

Accuracy Red Neuronal: 98.00%
[1m7/7[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m
10ms/step

Matriz de Confusión:
[[52  0  3]
 [ 1 76  0]
 [ 0  0 68]]

Reporte de Clasificación:
              precision    recall  f1-score   support

       BUENO       0.98      0.95      0.96        55
  INCORRECTO       1.00      0.99      0.99        77
    PERFECTO       0.96      1.00      0.98        68

    accuracy                           0.98       200
   macro avg       0.98      0.98      0.98       200
weighted avg       0.98      0.98      0.98       200
```

5. Gráfica de Rendimiento

Esta gráfica muestra si el modelo está aprendiendo correctamente o si presenta sobreajuste.

In [8]:
```
# In [6]:
# Gráfica de entrenamiento
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Entrenamiento')
plt.plot(history.history['val_accuracy'], label='Validación')
plt.title("Accuracy Red Neuronal")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```



En conclusión, la evaluación de la Red Neuronal Densa evidencia una sólida capacidad para modelar patrones no lineales y variaciones sutiles en los movimientos corporales del usuario. A diferencia del enfoque determinista del árbol, esta arquitectura ofrece una mayor robustez frente al ruido y a las inconsistencias naturales en la captura de coordenadas por visión artificial.

Las métricas de validación confirman que el modelo generaliza adecuadamente las posturas de baile sin caer en sobreajuste (overfitting). Esta flexibilidad algorítmica es clave para nuestro proyecto de Unexpo Dancing, ya que permite adaptar la detección a diferentes fisonomías y estilos de ejecución de los jugadores, elevando la precisión general del sistema de captura de movimiento.

In [ ]:

```

```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.